

FIBONACCI ALGORITHM (F(n))

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1.

Logic:

- Base Cases: If n is 0, return 0. If n is 1, return 1.
- Recursive Step: If n > 1, the function returns the sum of Fibonacci(n - 1) and Fibonacci(n - 2).

```
FUNCTION Fibonacci(n)
    // 1. Check for negative inputs
    IF n < 0 THEN
        PRINT "Error: Negative numbers not allowed"
        RETURN NULL

    // 2. Base Cases: Defining the start of the sequence
    IF n == 0 THEN
        RETURN 0
    ELSE IF n == 1 THEN
        RETURN 1

    // 3. Recursive Case: Branching into two previous states
    ELSE
        RETURN Fibonacci(n - 1) + Fibonacci(n - 2)
    END IF
END FUNCTION
```

```
def fibonacci(n):
    """Recursive function to calculate nth Fibonacci number."""
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

def main():
    try:
        num = int(input("Enter a non-negative integer for Fibonacci: "))
        if num < 0:
            print("Invalid input: Please enter a non-negative integer.")
        else:
            print(f"The Fibonacci number at position {num} is {fibonacci(num)}")
    except ValueError:
        print("Invalid input: Please enter a valid integer.")

if __name__ == "__main__":
    main()
```

1. FACTORIAL ALGORITHM (n!)

The factorial of a non-negative integer n is the product of all positive integers from 1 to n .

Logic:

- Base Case: If n is 0 or 1, return 1.
- Recursive Step: If $n > 1$, the function returns n multiplied by the factorial of $(n - 1)$. This continues until the base case is reached.

```
FUNCTION Factorial(n)
    // 1. Check for negative inputs to prevent infinite recursion
    IF n < 0 THEN
        PRINT "Error: Negative numbers not allowed"
        RETURN NULL

    // 2. Base Case: The stopping condition
    IF n == 0 OR n == 1 THEN
        RETURN 1

    // 3. Recursive Case: The function calls itself
    ELSE
        RETURN n * Factorial(n - 1)
    END IF
END FUNCTION
```

```
def factorial(n):
    """Recursive function to calculate factorial."""
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

def main():
    try:
        num = int(input("Enter a non-negative integer for Factorial: "))
        if num < 0:
            print("Invalid input: Please enter a non-negative integer.")
        else:
            print(f"The factorial of {num} is {factorial(num)}")
    except ValueError:
        print("Invalid input: Please enter a valid integer.")

if __name__ == "__main__":
    main()
```

PARINDROME

1. START

3. Pre-processing:

- Convert all characters in the string to lowercase to ensure the comparison is case-insensitive (e.g., 'Level' becomes 'level').

4. Array Transformation:

- Store the characters of the string into an array (list).

5. Reversal:

- Create a new array that contains the same characters but in reverse order.

6. Comparison:

- Compare the original character array with the reversed character array.

7. Decision:

- IF the original array is identical to the reversed array:
 - Output: "The word is a Palindrome."
- ELSE:
 - Output: "The word is a Normal word."

8. END

```
def check_palindrome(word):  
    # Convert word to lowercase to ensure case-insensitivity  
    word = word.lower()  
  
    # Convert string into an array (list) of characters  
    char_array = list(word)  
  
    # Create the reversed array  
    # The[::-1] syntax is the standard way to reverse a list in Python  
    reversed_array = char_array[::-1]  
  
    # Compare the two arrays  
    if char_array == reversed_array:  
        return True  
    else:  
        return False  
  
def main():  
    user_input = input("Enter a word to check if it is a palindrome: ")  
  
    if check_palindrome(user_input):  
        print(f'{user_input} is a Palindrome!')  
    else:  
        print(f'{user_input} is a Normal word.')  
  
if __name__ == "__main__":  
    main()
```

FACTORIAL





